```
"""
main.py
-------
Tkinter GUI for the AI Sentiment Analyzer
(Feature Set B).
Run with:  python main.py
"""

import tkinter as tk
from tkinter import messagebox

from sentiment_analyzer import
analyze_review, EMPTY_INPUT_MESSAGE

# ----------------------------------------
---------------------------------
# Colours and fonts
# ----------------------------------------
---------------------------------
BG_COLOR = "#f4f6fb"
HEADER_COLOR = "#2c3e90"
CARD_COLOR = "#ffffff"
TEXT_COLOR = "#222222"
MUTED_COLOR = "#777777"
SENTIMENT_COLORS = {
    "Positive": "#1e8e3e",
    "Negative": "#d93025",
    "Neutral": "#e37400",
}
```

```
FONT_TITLE = ("Segoe UI", 22, "bold")
FONT_NORMAL = ("Segoe UI", 11)
FONT_BOLD = ("Segoe UI", 11, "bold")
FONT_RESULT = ("Segoe UI", 16, "bold")

EXAMPLE_REVIEWS = {
    "Positive example": "I really loved
this product. It is excellent and
amazing.",
    "Negative example": "The product is
terrible and disappointing.",
    "Neutral example": "The product is
okay.",
}
PLACEHOLDER_TEXT =
EXAMPLE_REVIEWS["Positive example"]


class SentimentAnalyzerApp:
    """Builds the window and handles button
clicks."""

    def __init__(self, root):
        self.root = root
        self.placeholder_active = False

        root.title("AI Sentiment Analyzer")
        root.geometry("820x780")
        root.minsize(700, 700)
        root.configure(bg=BG_COLOR)

        self.build_header()
        self.build_input_section()
        self.build_result_section()
```

```
        self.show_placeholder()

    # ------------------------------------
---------------------------- UI
    def build_header(self):
        header = tk.Frame(self.root,
bg=HEADER_COLOR)
        header.pack(fill="x")

        tk.Label(header, text="AI Sentiment
Analyzer", font=FONT_TITLE,
                 bg=HEADER_COLOR,
fg="white").pack(pady=(15, 2))
        tk.Label(header,
                 text="Type a review and
find out whether it is Positive, "
                      "Negative or Neutral,
along with a sentiment score.",
                 font=FONT_NORMAL,
bg=HEADER_COLOR, fg="#d6dcff",
                 wraplength=760).pack(pady=
(0, 15))

    def build_input_section(self):
        card = tk.Frame(self.root,
bg=CARD_COLOR, padx=15, pady=12)
        card.pack(fill="x", padx=20, pady=
(15, 8))

        tk.Label(card, text="Enter Your
Review", font=FONT_BOLD,
                 bg=CARD_COLOR,
fg=TEXT_COLOR).pack(anchor="w")
```

```
        # Large multi-line review box with
a scrollbar
        text_frame = tk.Frame(card,
bg=CARD_COLOR)
        text_frame.pack(fill="x", pady=(6,
8))

        self.review_box =
tk.Text(text_frame, height=7, wrap="word",

font=FONT_NORMAL, relief="solid", bd=1,
                                    padx=8,
pady=8)
        scrollbar =
tk.Scrollbar(text_frame,
command=self.review_box.yview)

self.review_box.configure(yscrollcommand=sc
rollbar.set)
        self.review_box.pack(side="left",
fill="x", expand=True)
        scrollbar.pack(side="right",
fill="y")

        # Placeholder behaviour
        self.review_box.bind("<FocusIn>",
self.on_focus_in)
        self.review_box.bind("<FocusOut>",
self.on_focus_out)

        # Example review buttons
        example_row = tk.Frame(card,
bg=CARD_COLOR)
        example_row.pack(fill="x", pady=(0,
```

```
10))
        tk.Label(example_row, text="Try an
example:", font=("Segoe UI", 10),
                 bg=CARD_COLOR,
fg=MUTED_COLOR).pack(side="left", padx=(0,
8))
        for label, review in
EXAMPLE_REVIEWS.items():
            tk.Button(example_row,
text=label, font=("Segoe UI", 9),
                      relief="groove",
cursor="hand2",
                      command=lambda
r=review: self.load_example(r)
                      ).pack(side="left",
padx=3)

        # Main action buttons
        button_row = tk.Frame(card,
bg=CARD_COLOR)
        button_row.pack(fill="x")
        tk.Button(button_row, text="Analyze
Sentiment", font=FONT_BOLD,
                  bg=HEADER_COLOR,
fg="white", activebackground="#1f2d6e",
                  activeforeground="white",
relief="flat", padx=20, pady=8,
                  cursor="hand2",
command=self.on_analyze
                  ).pack(side="left")
        tk.Button(button_row, text="Clear",
font=FONT_BOLD, bg="#e0e0e0",

activebackground="#c8c8c8", relief="flat",
```

```
padx=20,
                  pady=8, cursor="hand2",
command=self.on_clear
                  ).pack(side="left",
padx=10)

    def build_result_section(self):
        card = tk.Frame(self.root,
bg=CARD_COLOR, padx=15, pady=12)
        card.pack(fill="both", expand=True,
padx=20, pady=(8, 20))

        tk.Label(card, text="Result", font=
("Segoe UI", 14, "bold"),
                 bg=CARD_COLOR,
fg=HEADER_COLOR).pack(anchor="w")

        # Entered review (read-only box)
        tk.Label(card, text="Entered
Review:", font=FONT_BOLD,
                 bg=CARD_COLOR,
fg=TEXT_COLOR).pack(anchor="w", pady=(8,
2))
        self.result_review = tk.Text(card,
height=3, wrap="word",

font=FONT_NORMAL, bg="#f7f7f7",

relief="flat", padx=6, pady=6,

state="disabled")
        self.result_review.pack(fill="x")

        # Sentiment classification
```

```
        row = tk.Frame(card, bg=CARD_COLOR)
        row.pack(fill="x", pady=(10, 0))
        tk.Label(row, text="Sentiment:",
font=FONT_BOLD, bg=CARD_COLOR,

fg=TEXT_COLOR).pack(side="left")
        self.sentiment_label =
tk.Label(row, text="-", font=FONT_RESULT,

bg=CARD_COLOR, fg=MUTED_COLOR)

self.sentiment_label.pack(side="left",
padx=10)

        # Sentiment score
        row = tk.Frame(card, bg=CARD_COLOR)
        row.pack(fill="x", pady=(4, 0))
        tk.Label(row, text="Sentiment
Score:", font=FONT_BOLD,
                 bg=CARD_COLOR,
fg=TEXT_COLOR).pack(side="left")
        self.score_label = tk.Label(row,
text="-", font=FONT_RESULT,

bg=CARD_COLOR, fg=MUTED_COLOR)
        self.score_label.pack(side="left",
padx=10)
        tk.Label(row, text="(range: -1.00
to +1.00)", font=("Segoe UI", 9),
                 bg=CARD_COLOR,
fg=MUTED_COLOR).pack(side="left")

        # Explanation
        tk.Label(card, text="Explanation:",
```

```
font=FONT_BOLD, bg=CARD_COLOR,

fg=TEXT_COLOR).pack(anchor="w", pady=(10,
2))
        self.explanation_label =
tk.Label(card, text="-", font=FONT_NORMAL,

bg=CARD_COLOR, fg=TEXT_COLOR,

wraplength=740, justify="left")

self.explanation_label.pack(anchor="w")

    # ------------------------------------
------------------ placeholder
    def show_placeholder(self):
        self.review_box.delete("1.0",
"end")
        self.review_box.insert("1.0",
PLACEHOLDER_TEXT)

self.review_box.config(fg=MUTED_COLOR)
        self.placeholder_active = True

    def on_focus_in(self, _event=None):
        if self.placeholder_active:
            self.review_box.delete("1.0",
"end")

self.review_box.config(fg=TEXT_COLOR)
            self.placeholder_active = False

    def on_focus_out(self, _event=None):
        if not self.review_box.get("1.0",
```

```
"end").strip():
            self.show_placeholder()

    def get_review_text(self):
        """Return the typed review ('' if
only the placeholder is shown)."""
        if self.placeholder_active:
            return ""
        return self.review_box.get("1.0",
"end").strip()

    # ------------------------------------
----------------- result helpers
    def set_result_review(self, text):

self.result_review.config(state="normal")
        self.result_review.delete("1.0",
"end")
        self.result_review.insert("1.0",
text)

self.result_review.config(state="disabled")

    def reset_result(self):
        self.set_result_review("")
        self.sentiment_label.config(text="-
", fg=MUTED_COLOR)
        self.score_label.config(text="-",
fg=MUTED_COLOR)

self.explanation_label.config(text="-",
fg=TEXT_COLOR)

    # ------------------------------------
```

```
---------------------- handlers
    def load_example(self, review):
        self.placeholder_active = False

self.review_box.config(fg=TEXT_COLOR)
        self.review_box.delete("1.0",
"end")
        self.review_box.insert("1.0",
review)
        self.reset_result()

    def on_analyze(self):
        """Called when 'Analyze Sentiment'
is clicked."""
        review = self.get_review_text()
        try:
            result = analyze_review(review)
        except ValueError:
            # Empty input
            self.reset_result()

self.explanation_label.config(text=EMPTY_IN
PUT_MESSAGE,

fg=SENTIMENT_COLORS["Negative"])
            messagebox.showwarning("Empty
Review", EMPTY_INPUT_MESSAGE)
            return
        except Exception as error:  #
unexpected problem - never crash the GUI
            messagebox.showerror("Error",
f"Something went wrong:\n{error}")
            return
```

```
        color =
SENTIMENT_COLORS[result["sentiment"]]

self.set_result_review(result["review"])

self.sentiment_label.config(text=result["se
ntiment"], fg=color)
        self.score_label.config(text=f"
{result['score']:+.2f}", fg=color)

self.explanation_label.config(text=result["
explanation"],

fg=TEXT_COLOR)

    def on_clear(self):
        """Called when 'Clear' is
clicked."""
        self.show_placeholder()
        self.reset_result()
        self.root.focus_set()


def main():
    root = tk.Tk()
    SentimentAnalyzerApp(root)
    root.mainloop()


if __name__ == "__main__":
    main()
```